МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ

про виконання лабораторної роботи №5

з курсу «Професійний Python»

«Обробка помилок, файли та серіалізація»

Виконав:
студент 3 курсу, групи ФЕП-31с
Гарбіч Данило

Перевірив:
професор
Свелеба Сергій Андрійович

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №5. «Обробка помилок, файли та серіалізація».

3. Мета роботи

Сформувати практичні навички створення надійних Python-програм, які коректно працюють із помилками, файловими ресурсами, конфігурацією та структурованими форматами даних. Опанувати механізм exceptions, конструкції `try/except/else/finally`, створення domain-specific ієрархій винятків, exception chaining (`raise ... from ...`), принципи EAFP і fail-fast, контекстні менеджери, промислове логування через `logging`, модуль `pathlib`, серіалізацію у JSON/YAML, потокову обробку даних (streaming I/O) та атомарний запис файлів (atomic write).

4. Номер індивідуального варіанта

Варіант №9: «Імпорт та експорт спортивних результатів».

5. Постановка задачі

Створити надійний модуль імпорту, валідації, фільтрації та експорту спортивних результатів відповідно до варіанта №9:

- Поля сутності: `athlete`, `sport`, `result`, `category`, `timestamp`.
- Працювати виключно через сучасний модуль `pathlib.Path`.
- Реалізувати потокове читання даних (Streaming input через генератори) для запобігання вичерпанню RAM.
- Підтримувати імпорт з CSV, JSON Lines та YAML через єдиний абстрактний інтерфейс (протокол `Importer`).
- Забезпечити конфігурування конвеєра через зовнішній файл `config.yaml` із валідацією параметрів (fail-fast).
- Побудувати domain-specific ієрархію винятків (`ApplicationError` → `DataError` → `RecordValidationError`).
- Використати ланцюжки винятків (`raise ... from error`) для збереження першопричини низкорівневих помилок.
- Реалізувати власний контекстний менеджер для транзакційності та профілювання часу операцій (`logged_operation`).
- Налаштувати багаторівневе логування з очищенням чутливих даних.
- Підтримувати дві політики надійності: `strict` (переривання) та `tolerant` (пропуск помилок із реєстрацією у звіті).
- Реалізувати атомарний експорт (atomic write через `*.tmp` та `replace`).

- Зібрати статистику обробки (`total`, `valid`, `invalid`, `exported`) та сформувати окремий звіт про помилки.

6. Input data schema

Схема полів для індивідуального варіанта 9:

- `athlete` (рядок): Ім'я спортсмена (непорожній рядок без провідних/кінцевих пробілів).
- `sport` (рядок): Назва спортивної дисципліни (повинна входити до списку `allowed_sports` із конфігурації).
- `result` (дійсне число): Числовий результат (перевіряється на відповідність діапазону `[min_value, max_value]` для конкретного виду спорту).
- `category` (рядок): Категорія («Дорослі», «Юніори», «Молодь», «Ветерани»).
- `timestamp` (рядок/дата): Часова мітка фіксації у форматі ISO 8601 (не старіша за `min_timestamp`).

7. Приклад вхідного CSV (`data/sports_results.csv`)

```
athlete,sport,result,category,timestamp
Ярослава Магучіх,Стрибки у висоту,2.10,Дорослі,2024-07-07T18:30:00
Ірина Геращенко,Стрибки у висоту,2.00,Дорослі,2024-08-04T19:00:00
Юлія Левченко,Стрибки у висоту,1.95,Дорослі,2024-06-15T17:45:00
,Стрибки у висоту,1.90,Дорослі,2024-07-10T12:00:00
Ілля Ковтун,Спортивна гімнастика,15.65,Дорослі,2024-07-29T16:15:00
Олег Верняєв,Спортивна гімнастика,15.30,Дорослі,2024-07-29T16:30:00
Михайло Романчук,Плавання 800м,461.50,Дорослі,2024-07-30T11:20:00
Олександр Желтяков,Плавання 800м,472.10,Юніори,2024-07-30T11:25:00
Сергій Бубка,Стрибки у висоту,3.80,Ветерани,2024-05-10T14:00:00
Владислав Лавський,Стрибки з жердиною,5.80,Дорослі,2024-06-20T18:00:00
Артем Коваль,Біг 100м,invalid_num,Юніори,2024-08-10T10:00:00
Олена Старікова,Велоспорт,10.75,НевідомаКатегорія,2024-07-12T15:00:00
Марія Савченко,Стрільба з лука,680.00,Дорослі,2020-01-01T12:00:00
Андрій Проценко,Стрибки у висоту,2.25,Дорослі,2024-08-01T19:30:00
```

8. Структура проєкту

```
c:\labs\Lab_05\
├── pyproject.toml                      # Конфігурація проєкту та муру
├── config.yaml                         # Зовнішня конфігурація YAML
├── README.md                           # Опис проєкту
├── data\
│   └── sports_results.csv              # Вхідні дані (валідно-дефектний набір)
├── output\
│   ├── sports_results.json             # Атомарний експорт валідних записів
│   ├── error_report.json               # Звіт відхилених записів (JSON)
│   └── error_report.csv                # Звіт відхилених записів (CSV)
├── logs\
│   └── application.log                 # Файловий журнал виконання
├── src\reliable_sports_importer\
│   ├── __init__.py                     # Версія пакета
│   ├── py.typed                        # Маркер типізації PEP 561
│   ├── exceptions.py                   # Доменна ієрархія винятків
│   ├── models.py                       # Датакласи конфігурації та результатів
│   ├── config.py                       # Завантажувач та валідатор конфігурації
│   ├── logging_config.py               # Налаштування багаторівневого логування
│   ├── utils.py                        # Власний context manager та atomic writer
│   ├── readers.py                      # Потокові імпортери (CSV, JSONL, YAML)
│   ├── validators.py                   # Валідація правил предметної області
│   ├── exporters.py                    # Атомарні експортери даних і помилок
│   ├── services.py                     # Оркестратор пайплайну та збір статистики
│   └── main.py                         # Точка входу в програму
└── tests\
    ├── test_sports_importer.py         # Модульні тести бізнес-правил
    └── experiments_runner.py           # Автоматизований запуск 5 експериментів
```

9. Ієрархія винятків (`exceptions.py`)

```
class ApplicationError(Exception):
    """Базовий виняток додатку."""

class ConfigurationError(ApplicationError):
    """Виникає при помилках конфігурації."""

class DataError(ApplicationError):
    """Базовий виняток для помилок обробки даних."""

class DataImportError(DataError):
    """Критичні помилки відкриття чи парсингу вхідного потоку."""

class RecordValidationError(DataError):
    """Помилка валідації окремого запису предметної області."""
    def __init__(self, message: str, *, line_number: int | None = None,
                 field: str | None = None, raw_value: Any = None) -> None:
        super().__init__(message)
        self.line_number = line_number
        self.field = field
        self.raw_value = raw_value

class DataExportError(DataError):
    """Помилка атомарного експорту або серіалізації."""
```

10. Пояснення custom exceptions

Власна ієрархія винятків дозволяє чітко розмежовувати технічні низькорівневі збої операційної системи та помилки семантики даних. Спеціалізований клас `RecordValidationError` містить контекстні атрибути (номер рядка, ім'я поля, дефектне значення), що дозволяє сервісу точно ідентифікувати причину браку без деструктивного переривання всього циклу імпорту в tolerant-режимі.

11. Приклад exception chaining

```
result_raw = row.get("result")
try:
    result_val = float(result_raw)
except (TypeError, ValueError) as error:
    raise RecordValidationError(
        f"Результат має бути дійсним числовим значенням: {result_raw}",
        line_number=line_number,
        field="result",
        raw_value=result_raw,
    ) from error
```

12. Вміст `config.yaml`

```yaml
input:
  path: data/sports_results.csv

output:
  path: output/sports_results.json
  error_report_path: output/error_report.json

processing:
  skip_invalid: true
  min_timestamp: "2024-01-01T00:00:00"
  allowed_sports:
    - "Стрибки у висоту"
    - "Плавання 800м"
    - "Спортивна гімнастика"
    - "Біг 100м"
    - "Стрільба з лука"
  allowed_categories:
    - "Дорослі"
    - "Юніори"
    - "Молодь"
    - "Ветерани"
  sport_rules:
    "Стрибки у висоту":
      min_value: 1.00
      max_value: 2.50
    "Плавання 800м":
      min_value: 400.00
      max_value: 800.00
    "Спортивна гімнастика":
      min_value: 10.00
      max_value: 16.50
    "Біг 100м":
      min_value: 9.50
      max_value: 20.00
    "Стрільба з лука":
      min_value: 0.00
      max_value: 720.00

logging:
  level: INFO
  path: logs/application.log
```

13. Код configuration loader та validation (`config.py`)

```
def load_config(path: Path) -> AppConfig:
    if not path.exists():
        raise ConfigurationError(f"Конфігураційний файл не знайдено: {path}")
    try:
        with path.open("r", encoding="utf-8") as file:
            raw: Any = yaml.safe_load(file)
    except yaml.YAMLError as error:
        raise ConfigurationError(f"Синтаксична помилка у YAML конфігурації: {erro
r}") from error
    except OSError as error:
        raise ConfigurationError(f"Неможливо прочитати файл конфігурації: {erro
r}") from error

    config = AppConfig(...)
    validate_config(config)
    return config

def validate_config(config: AppConfig) -> None:
    if not config.input_path.exists():
        raise ConfigurationError(f"Вхідний файл для обробки не знайдено: {config.i
nput_path}")
    if not config.processing.allowed_sports:
        raise ConfigurationError("Список дозволених видів спорту не може бути поро
жнім")
    for sport_name, rule in config.processing.sport_rules.items():
        if rule.min_value > rule.max_value:
            raise ConfigurationError(
                f"Некоректне правило для '{sport_name}': min_value ({rule.min_valu
e}) > max_value ({rule.max_value})"
            )
```

14. Logging configuration (`logging_config.py`)

```
def configure_logging(level: str, path: Path) -> None:
    path.parent.mkdir(parents=True, exist_ok=True)
    numeric_level = getattr(logging, level.upper(), logging.INFO)
    file_handler = logging.FileHandler(path, encoding="utf-8")
    stream_handler = logging.StreamHandler()
    formatter = logging.Formatter(
        fmt="%(asctime)s | %(levelname)-7s | %(name)s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )
    file_handler.setFormatter(formatter)
    stream_handler.setFormatter(formatter)
    root_logger = logging.getLogger()
    root_logger.setLevel(numeric_level)
    root_logger.handlers = [file_handler, stream_handler]
```

15. Фрагмент log file (`logs/application.log`)

```
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.main | ============================
====================================
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.main | ЛАБОРАТОРНА РОБОТА №5: ОБРО
БКА ПОМИЛОК, ФАЙЛИ ТА СЕРІАЛІЗАЦІЯ
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.main | Варіант 9: Імпорт та експор
т спортивних результатів
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.main | Конфігурацію успішно завант
ажено з: config.yaml
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.main | Режим обробки: Tolerant (sk
ip_invalid=True)
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.utils | Початок операції: Sports D
ata Pipeline Processing
2026-10-18 19:07:20 | WARNING  | reliable_sports_importer.services | Некоректний запис на ря
дку 5: Ім'я спортсмена не може бути порожнім (рядок 5, поле 'athlete', значення='')
2026-10-18 19:07:20 | WARNING  | reliable_sports_importer.services | Некоректний запис на ря
дку 10: Результат 3.8 виходить за межі допустимого діапазону [1.0, 2.5] для виду спорту 'Ст
рибки у висоту' (рядок 10, поле 'result', значення='3.8')
2026-10-18 19:07:20 | WARNING  | reliable_sports_importer.services | Некоректний запис на ря
дку 11: Вид спорту 'Стрибки з жердиною' не входить до переліку дозволених
2026-10-18 19:07:20 | WARNING  | reliable_sports_importer.services | Некоректний запис на ря
дку 12: Результат має бути дійсним числовим значенням: invalid_num
2026-10-18 19:07:20 | WARNING  | reliable_sports_importer.services | Некоректний запис на ря
дку 13: Вид спорту 'Велоспорт' не входить до переліку дозволених
2026-10-18 19:07:20 | WARNING  | reliable_sports_importer.services | Некоректний запис на ря
дку 14: Часова мітка 2020-01-01 12:00:00 є старішою за мінімально дозволену
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.services | Звіт про некоректні зап
иси експортовано у output\error_report.json та output\error_report.csv
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.utils | Операція 'Sports Data Pipe
line Processing' успішно завершена за 0.004240 с
2026-10-18 19:07:20 | INFO     | reliable_sports_importer.main | Всього прочитано записів: 1
4 | Валідних: 8 | Некоректних: 6 | Експортовано: 8
```

16. Streaming reader (`readers.py`)

```
class CsvImporter:
    def read(self, path: Path) -> Iterator[tuple[int, dict[str, Any]]]:
        try:
            with path.open("r", encoding="utf-8", newline="") as file:
                reader = csv.DictReader(file)
                for line_number, row in enumerate(reader, start=2):
                    yield line_number, row
        except OSError as error:
            raise DataImportError(f"Помилка відкриття CSV файлу '{path}': {erro
r}") from error
```

17. Validation code (`validators.py`)

```
def parse_and_validate_record(row: dict[str, Any], *, line_number: int, config: Pr
ocessingConfig) -> SportsResult:
    athlete_raw = str(row.get("athlete", "")).strip()
    if not athlete_raw:
        raise RecordValidationError("Ім'я спортсмена не може бути порожнім", line_
number=line_number, field="athlete")

    sport_raw = str(row.get("sport", "")).strip()
    if config.allowed_sports and sport_raw not in config.allowed_sports:
        raise RecordValidationError(f"Вид спорту '{sport_raw}' не дозволений", lin
e_number=line_number, field="sport")

    try:
        result_val = float(row.get("result"))
    except (TypeError, ValueError) as error:
        raise RecordValidationError("Результат має бути числовим", line_number=lin
e_number, field="result") from error

    rule = config.sport_rules.get(sport_raw)
    if rule and not (rule.min_value <= result_val <= rule.max_value):
        raise RecordValidationError(f"Результат {result_val} виходить за межі діап
азону", line_number=line_number, field="result")

    return SportsResult(athlete=athlete_raw, sport=sport_raw, result=result_val, c
ategory=row["category"], timestamp=datetime.fromisoformat(row["timestamp"]))
```

18. Strict vs Tolerant processing (`services.py`)

```
for line_number, raw_row in importer.read(self.config.input_path):
    self.statistics.total_records += 1
    try:
        sports_result = parse_and_validate_record(raw_row, line_number=line_numbe
r, config=self.config.processing)
    except RecordValidationError as error:
        self.statistics.invalid_records += 1
        self.invalid_records.append(InvalidRecordDetail(line_number, str(raw_row),
str(error), error.field))
        logger.warning("Некоректний запис на рядку %s: %s", line_number, error)
        if self.config.processing.skip_invalid:
            continue
        raise
    self.statistics.valid_records += 1
    yield sports_result
```

19. JSON serialization та Atomic export (`utils.py`, `exporters.py`)

```
@contextmanager
def atomic_file_writer(target_path: Path, encoding: str = "utf-8") -> Iterator[An
y]:
    target_path.parent.mkdir(parents=True, exist_ok=True)
    temp_path = target_path.with_suffix(target_path.suffix + ".tmp")
    try:
        with temp_path.open("w", encoding=encoding) as file:
            yield file
        temp_path.replace(target_path)
    except Exception as error:
        temp_path.unlink(missing_ok=True)
        raise DataExportError(f"Неможливо виконати атомарний запис у {target_pat
h}: {error}") from error
```

20. Власний context manager (`utils.py`)

```
@contextmanager
def logged_operation(operation_name: str) -> Iterator[None]:
    logger.info("Початок операції: %s", operation_name)
    start_time = perf_counter()
    try:
        yield
    except Exception as error:
        elapsed = perf_counter() - start_time
        logger.exception("Операція '%s' зазнала аварійного збою після %.6f с: %s",
operation_name, elapsed, error)
        raise
    else:
        elapsed = perf_counter() - start_time
        logger.info("Операція '%s' успішно завершена за %.6f с", operation_name, e
lapsed)
```

21. Результат виконання програми (Screenshot execution)

```
============================================================
ЛАБОРАТОРНА РОБОТА №5: ОБРОБКА ПОМИЛОК, ФАЙЛИ ТА СЕРІАЛІЗАЦІЯ
Варіант 9: Імпорт та експорт спортивних результатів
Конфігурацію успішно завантажено з: config.yaml
Режим обробки: Tolerant (skip_invalid=True)
============================================================
Початок операції: Sports Data Pipeline Processing
WARNING: Некоректний запис на рядку 5: Ім'я спортсмена не може бути порожнім
WARNING: Некоректний запис на рядку 10: Результат 3.8 виходить за межі допустимого діапазон
у [1.0, 2.5]
WARNING: Некоректний запис на рядку 11: Вид спорту 'Стрибки з жердиною' не входить до перел
іку дозволених
WARNING: Некоректний запис на рядку 12: Результат має бути дійсним числовим значенням: inva
lid_num
WARNING: Некоректний запис на рядку 13: Вид спорту 'Велоспорт' не входить до переліку дозво
лених
WARNING: Некоректний запис на рядку 14: Часова мітка 2020-01-01 12:00:00 є старішою за міні
мально дозволену
INFO: Звіт про некоректні записи експортовано у output\error_report.json та output\error_re
port.csv
Операція 'Sports Data Pipeline Processing' успішно завершена за 0.004240 с
------------------------------------------------------------
ПІДСУМКОВА СТАТИСТИКА ВИКОНАННЯ:
  Всього прочитано записів: 14
  Валідних записів (успішно): 8
  Некоректних записів (помилки): 6
  Експортовано у output\sports_results.json: 8
  Окремий звіт про помилки записано у: output\error_report.json
------------------------------------------------------------
```

22. Результати Експерименту 1 (Strict vs Tolerant)

Параметр	Strict Mode (skip_invalid: false)	Tolerant Mode (skip_invalid: true)
Оброблено записів	1001 (зупинено на 1-му невалідному)	1010 (повний набір)
Valid	1000	1000
Invalid	1	10
Exported	0 (транзакцію відхилено)	1000
Завершення application	Перервано винятком DataExportError	Успішно з кодом 0

23. Результати Експерименту 2 (Streaming vs Eager)

Кількість записів	Eager time (c)	Streaming time (c)	Eager peak RAM (MB)	Streaming peak RAM (MB)
10 000	0.1112	0.0804	4.92 MB	0.0656 MB
100 000	0.8792	0.8077	48.94 MB	0.0653 MB
500 000	4.4826	4.0566	245.17 MB	0.0653 MB

Висновки з експерименту 2: Генератори забезпечують стабільне константне споживання пам'яті $O(1)$ на рівні ~0.065 MB незалежно від кількості записів (від 10 000 до 500 000). У той же час eager-підхід вимагає лінійного зростання пам'яті $O(N)$ (до 245 MB), що призведе до аварійного вичерпання ресурсів машини при обробці багатогігабайтних CSV.

24. Результати Експерименту 3 (Exception Chaining)

```
Traceback (most recent call last):
  File "C:\labs\Lab_05\src\reliable_sports_importer\validators.py", line 47, in parse_and_v
alidate_record
    result_val = float(result_raw)
                 ^^^^^^^^^^^^^^^^
ValueError: could not convert string to float: 'not_a_number'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "C:\labs\Lab_05\tests\experiments_runner.py", line 200, in run_experiment_3
    parse_and_validate_record(bad_row, line_number=42, config=dummy_cfg)
  File "C:\labs\Lab_05\src\reliable_sports_importer\validators.py", line 49, in parse_and_v
alidate_record
    raise RecordValidationError(
reliable_sports_importer.exceptions.RecordValidationError: Результат має бути дійсним число
вим значенням: not_a_number (рядок 42, поле 'result', значення='not_a_number')
```

25. Результати Експерименту 4 (Atomic Output)

Під час симуляції аварійного збою всередині контекстного менеджера `atomic_file_writer` було зафіксовано такі результати:

- Попередній валідний файл `output/atomic_test_production.json` залишився повністю неушкодженим (True).
- Тимчасовий файл `output/atomic_test_production.json.tmp` було гарантовано видалено з диска блоком `except` (True).
- Виняток було коректно перетворено на `DataExportError` із збереженням першопричини (True).

26. Результати Експерименту 5 (Configuration Errors / Fail-fast)

№	Сценарій збою	Реакція системи (Fail-fast)
1	Відсутній файл `config.yaml`	`ConfigurationError: Конфігураційний файл не знайдено: config_non_existent.yaml`
2	Неправильний синтаксис YAML	`ConfigurationError: Синтаксична помилка у YAML конфігурації: while parsing a flow sequence...`
3	Відсутня секція `input`	`ConfigurationError: Некоректна структура конфігурації: 'input'`
4	Некоректне правило `min_value > max_value`	`ConfigurationError: Некоректне правило для 'Стрибки': min_value (300.0) > max_value (10.0)`
5	Вхідний файл відсутній на диску	`ConfigurationError: Вхідний файл для обробки не знайдено: missing_file.csv`
6	Порожній список дозволених видів спорту	`ConfigurationError: Список дозволених видів спорту (allowed_sports) не може бути порожнім`

27. Відповіді на контрольні запитання (70 запитань)

1. Що таке exception?
Об'єкт у Python, що сигналізує про виникнення помилки чи нештатної ситуації під час виконання коду та змінює нормальний потік керування.

2. Для чого використовується try?
Для позначення захищеного блоку коду, всередині якого можливе виникнення винятків.

3. Для чого except?
Для перехоплення та обробки винятків заданого типу, які виникли всередині блоку try.

4. Коли виконується else після try?
Коли блок try завершився повністю успішно без виникнення жодних винятків.

5. Коли виконується finally?
Завжди: незалежно від того, сталася помилка, чи блок завершився штатно.

6. Для чого використовується raise?
Для явного збудження екземпляра винятку розробником.

7. Що означає raise без аргументів?
Повторне підняття (re-raise) поточного активного винятку з блоку except зі збереженням контексту.

8. Чому не рекомендується except: pass?
Маскує програмні дефекти (помилки синтаксису, typo у змінних) та блокує системні переривання (KeyboardInterrupt).

9. Чому потрібно перехоплювати конкретні exception types?

Щоб обробляти лише очікувані нештатні ситуації, залишаючи невідомі збої для виявлення та налагодження.

10. Що таке exception hierarchy?

Дерево наслідування класів винятків із спільними абстракціями на вершині (наприклад, BaseException → Exception → ApplicationError).

11. Для чого створювати custom exceptions?

Для надання помилкам предметного змісту, додавання структурованих атрибутів (рядок, поле) та точкового перехоплення.

12. Що таке exception chaining?

Механізм зв'язування винятків для збереження первинного контексту низькорівневої помилки.

13. Що означає raise ... from ...?

Явне зв'язування винятків, що зберігає початковий виняток у полі __cause__.

14. Яку перевагу дає chaining для debugging?

Відображає повну історію виникнення проблеми у traceback: від технічної помилки бібліотеки до доменного збою.

15. Що означає EAFP?

«Easier to Ask for Forgiveness than Permission» — спочатку виконуємо операцію, а в разі збою перехоплюємо виняток.

16. Що означає LBYL?

«Look Before You Leap» — перед виконанням перевіряємо всі умови за допомогою if.

17. Що таке fail-fast?

Підхід, за якого система падає одразу при виявленні невалідного стану, не починаючи ресурсномістких операцій.

18. Де доцільно застосовувати fail-fast?

При завантаженні конфігурацій, валідації параметрів підключення до БД та ініціалізації сервісів.

19. Що таке context manager?

Об'єкт, що визначає контекст виконання для оператора with через методи __enter__ та __exit__.

20. Для чого використовується with?

Для автоматичного та гарантованого управління життєвим циклом ресурсів (файли, блокування, з'єднання).

21. Які methods утворюють context manager protocol?

Методи __enter__() та __exit__().

22. Що робить __enter__()?
Ініціалізує ресурс та повертає його у блок with.

23. Що робить __exit__()?
Звільняє ресурс та обробляє можливий виняток.

24. Чи закриває with файл при exception?
Так, інтерпретатор гарантовано викликає __exit__ навіть при аварійному виході.

25. Для чого contextlib.contextmanager?
Декоратор для зручного створення контекстних менеджерів за допомогою генератора з yield.

26. Яку роль має yield у generator-based context manager?
Розділяє фазу ініціалізації ресурсу та фазу його обов'язкового закриття у finally.

27. Що таке pathlib.Path?
Об'єктно-орієнтоване представлення шляху у файловій системі в сучасному Python.

28. Чому pathlib кращий за ручну конкатенацію шляхів?
Він гарантує кросплатформеність (роздільники / та \), підтримує оператор / та має безпечний API.

29. Для чого задається encoding="utf-8"?
Для усунення залежності від системного локального кодування операційної системи (наприклад CP1251).

30. Що таке CSV?
Текстовий табличний формат з роздільниками-комами.

31. Для чого csv.reader?
Для побудкового читання CSV у формі списків рядків list[str].

32. Для чого csv.DictReader?
Для читання CSV у форматі словників dict[str, str] із заголовками як ключами.

33. Чим DictReader зручний для structured data?
Дозволяє звертатися за іменами колонок замість індексів, що запобігає помилкам при зміні порядку колонок.

34. Для чого csv.DictWriter?
Для запису словників у файл CSV із автоматичним формуванням заголовків.

35. Що таке JSON?
Текстовий формат обміну структурованими даними на базі масивів та ключів-значень.

36. Що таке serialization?
Перетворення внутрішніх об'єктів пам'яті у формат зберігання або передачі (наприклад у JSON).

37. Що таке deserialization?

Зворотне відновлення об'єктів мови програмування з текстового чи байтового представлення.

38. Чим json.dump() відрізняється від json.dumps()?

dump() пише дані прямо у файл, а dumps() повертає рядок str.

39. Чим json.load() відрізняється від json.loads()?

load() зчитує та парсить вміст файлу, а loads() парсить рядок із пам'яті.

40. Що таке JSONDecodeError?

Виняток, що сигналізує про синтаксичну некоректність документа JSON.

41. Для чого використовується YAML?

Зручний людино-читабельний формат для складних вкладених конфігурацій.

42. Чому рекомендується yaml.safe_load()?

Запобігає виконанню довільного коду з ненадійних джерел через теги Python.

43. Які типові сценарії використання JSON?

Взаємодія через REST API, клієнт-серверний зв'язок, передача payload.

44. Які типові сценарії використання YAML?

Файли налаштувань програм, CI/CD конфігурації, Kubernetes деплої.

45. Що таке configuration file?

Зовнішній файл для зміни параметрів програми без зміни її двійкового чи вихідного коду.

46. Чому configuration краще не hard-code у Python?

Щоб мати змогу змінювати поведінку в різних оточеннях (dev/prod) без повторного розгортання коду.

47. Що потрібно перевіряти у configuration?

Існування шляхів, діапазони чисел, типи даних та обов'язкові ключі.

48. Що таке logging?

Системний запис подій програми з рівнями пріоритету та контекстними даними.

49. Чим logging відрізняється від print()?

Надає рівні важливості, додає часові мітки, пише у файл і легко фільтрується.

50. Які існують logging levels?

DEBUG, INFO, WARNING, ERROR, CRITICAL.

51. Для чого logger.info()?

Для фіксації штатних очікуваних подій ходу виконання.

52. Для чого logger.warning()?

Для сповіщення про потенційні проблеми чи дефекти даних, які не зупиняють роботу.

53. Для чого logger.error()?
Для запису інформації про збої, через які операція не була виконана.

54. Для чого logger.exception()?
Логує повідомлення рівня ERROR і автоматично додає повний traceback.

55. Що містить traceback?
Список активних викликів функцій від точки падіння до верхнього рівня стеку.

56. Чому secrets не можна записувати в logs?
Через загрозу витоку паролів, API ключів та конфіденційних персональних даних.

57. Що таке streaming I/O?
Обробка потоку даних по одному елементу без утримання всього масиву в RAM.

58. Яка перевага streaming для великих files?
Фіксоване споживання оперативної пам'яті O(1) запобігає OutOfMemory.

59. Що таке eager file loading?
Повне зчитування всього вмісту файлу в пам'ять перед початком його аналізу.

60. Чому list(reader) може бути небажаним?
Він матеріалізує всі записи файлу, що аварійно вичерпує пам'ять на великих наборах.

61. Що таке strict validation mode?
Режим, де будь-яка неточність призводить до негайного падіння всієї операції.

62. Що таке tolerant mode?
Режим, де некоректні записи відкидаються або логуються, а валідні — продовжують обробку.

63. Коли доцільно використовувати strict mode?
У фінансових, банківських або медичних системах з нульовою толерантністю до помилок.

64. Коли tolerant mode?
В аналітиці великих даних, логуванні, парсингу відкритих джерел інтернету.

65. Що таке atomic write?
Запис у тимчасовий файл із наступною миттєвою заміною цільового файлу лише у разі успіху.

66. Для чого використовується temporary file?
Щоб зберегти попередній цільовий файл від пошкодження під час запису нового.

67. Що може статися при звичайному output write та аварійному завершенні?
Цільовий файл буде обрізано (corrupted), а попередні робочі дані — безповоротно втрачено.

68. Навіщо видаляти temporary file після error?
Щоб очистити дисковий простір та не залишати сміттєвих незавершених файлів.

69. Що таке JSON Lines?

Формат запису, в якому кожен рядок файлу є окремим валідним JSON-об'єктом.

70. Чому JSON Lines зручний для streaming?

Його можна читати і писати построково без необхідності парсити квадратні дужки всього масиву.

28. Висновок

Під час виконання лабораторної роботи №5 було успішно розроблено надійний модуль імпорту, валідації та експорту спортивних результатів відповідно до варіанта №9:

1. Спроектовано власну ієрархію винятків (`ApplicationError`, `DataError`, `RecordValidationError`) із передачею контексту помилок та збереженням низькорівневих причин через механізм `exception chaining` (`from error`).
2. Побудовано конвеєр потокової обробки на основі генераторів (Streaming I/O), що забезпечило стабільне споживання оперативної пам'яті $O(1)$ (~0.065 MB) навіть при обробці 500 000 записів, запобігаючи помилкам вичерпання пам'яті.
3. Реалізовано надійний контекстний менеджер атомарного запису (`atomic_file_writer`), який унеможливлює пошкодження цільових вихідних файлів при раптових аварійних зупинках.
4. Забезпечено гнучке конфігурування конвеєра через зовнішній YAML-файл із суворою перевіркою параметрів на етапі запуску (fail-fast).
5. Кодова база пройшла перевірку статичним типізатором `mypy` в суворому режимі (`strict = true`) без жодних зауважень.